DEVELOPMENT GUIDE

Hospitality Hackathon 2026

Website Update Project

Setup, Workflow & Deployment Guide

Prepared for: Bisrat Gizaw

Repository: github.com/Abrish-21/alx-hackathon-web

March 2026

1. Prerequisites — What You Need Installed

Before touching any code, make sure you have the following tools installed on your computer. These are the building blocks that make everything work.

1.1 Node.js and npm

Node.js is the JavaScript runtime that lets you run the project locally. **npm** (Node Package Manager) comes bundled with Node.js and is used to install project dependencies (libraries the project needs).

How to install:

1. Go to https://nodejs.org
2. Download the LTS (Long Term Support) version — this is the stable, recommended one
3. Run the installer and follow the prompts (defaults are fine)

Verify it worked:

```
node --version
npm --version
```

Both commands should print a version number (e.g., v20.11.0 and 10.2.4). If you get an error, Node.js is not installed correctly.

1.2 Git

Git is the version control system that tracks every change to the code. It lets multiple people work on the same project without overwriting each other's work.

How to install:

- macOS: Open Terminal and run: xcode-select --install
- Windows: Download from https://git-scm.com and run the installer
- Linux: Run: sudo apt install git

Verify it worked:

```
git --version
```

Should print something like: git version 2.43.0

1.3 A Code Editor

You need a code editor to view and modify the project files. **Visual Studio Code (VS Code)** is the most popular free option and works great for this project.

How to install:

4. Go to https://code.visualstudio.com
5. Download and install for your operating system

Recommended VS Code extensions (optional but helpful):

- ES7+ React/Redux/React-Native snippets — speeds up writing React code
- Tailwind CSS IntelliSense — auto-completes Tailwind class names
- Prettier — automatically formats your code on save
- GitLens — shows who changed each line and when

1.4 A GitHub Account

You already have this (your username is **BisratJ**). Make sure you've accepted the collaboration invite from @Abrish-21 so you have write access to the repository.

> ⚠ **Important**
> If you haven't accepted the invite yet, go to https://github.com/Abrish-21/alx-hackathon-web/invitations while signed in as BisratJ.

2. Getting the Code onto Your Computer

Step 1: Open your terminal

- macOS: Open the Terminal app (search for it in Spotlight)
- Windows: Open Git Bash (installed with Git) or Command Prompt
- Linux: Open your terminal emulator

Step 2: Navigate to where you want the project

Choose a folder on your computer where you keep projects. For example:

```
cd ~/Documents
```

This moves you into your Documents folder. The project will be created as a subfolder inside here.

Step 3: Clone the repository

This downloads the entire project (all code, history, and files) from GitHub to your computer:

```
git clone https://github.com/Abrish-21/alx-hackathon-web.git
```

You should see output showing the download progress. When it finishes, a new folder called alx-hackathon-web will exist.

Step 4: Move into the project folder

```
cd alx-hackathon-web
```

Step 5: Install dependencies

This reads the package.json file and downloads every library the project needs:

```
npm install
```

This may take 1–2 minutes. You'll see a progress bar. When it finishes, a node_modules folder will appear. This folder is large and should never be committed to Git (it's already in .gitignore).

> **Success check**
> Run: ls node_modules (macOS/Linux) or dir node_modules (Windows). If it lists many folders, you're good.

3. Branching Strategy — How to Work Safely

This is the most important concept for making changes without breaking the live site. Think of branches like parallel timelines — you make all your changes in your own timeline, and only merge them into the “real” timeline when they’re tested and ready.

3.1 Understanding the Branch Structure

Branch	Purpose	Rule
main	The live site. Vercel deploys from this branch.	NEVER commit directly to main. All changes go through PRs.
2026-updates	Your main working branch for all 2026 changes.	This collects all your feature branches. Create a PR from here to main when ready.
feature/*	Individual task branches (e.g., feature/homepage-hero)	One branch per task. Merge into 2026-updates when complete.

3.2 Creating Your Working Branch

Run these commands right after cloning. You only need to do this once:

```
# Make sure you’re on main and it’s up to date
git checkout main
git pull origin main

# Create your working branch
git checkout -b 2026-updates

# Push it to GitHub so it exists remotely too
git push -u origin 2026-updates
```

Now you are on the 2026-updates branch. Everything you do from here is isolated from the live site.

3.3 Creating Feature Branches (For Each Task)

For each distinct task from the brief, create a separate feature branch off 2026-updates. This keeps changes organized and easy to review:

```
# Make sure you’re on 2026-updates
git checkout 2026-updates

# Create a feature branch for a specific task
git checkout -b feature/global-date-updates

# ... make your changes, test them ...

# When done, commit your work
git add -A
git commit -m "Update all dates and venues to 2026"
```

```
# Push the feature branch
git push -u origin feature/global-date-updates
```

Suggested feature branch names:

- feature/global-date-updates — Section 1.1 date and venue changes
- feature/homepage-hero — Section 2.1 hero section redesign
- feature/homepage-sections — Sections 2.3–2.6 (AI focus, speakers, judges, partners)
- feature/registration-restructure — Section 3 registration page
- feature/agenda-page — Section 4 agenda rewrite
- feature/past-events-page — Section 5 new 2025 page
- feature/faq-section — Section 6 FAQ
- feature/resources-training — Section 8 training program

> **Why feature branches?**
> If something goes wrong in one task, you can throw away just that branch without affecting your other completed work. It also makes code review much easier because each PR covers one focused change.

4. Running the Project Locally

4.1 Starting the Development Server

This is what you'll do every time you sit down to work:

```
# Navigate to the project folder (if not already there)
cd ~/Documents/alx-hackathon-web

# Make sure you're on the right branch
git checkout feature/your-current-task

# Start the development server
npm run dev
```

You'll see output like:

```
  VITE v5.x.x  ready in 500 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: http://192.168.1.x:5173/
```

Open **http://localhost:5173/** in your browser. This is your local version of the site. Every time you save a file, the browser will automatically reload with your changes — this is called **Hot Module Replacement (HMR)**.

4.2 Stopping the Server

Press **Ctrl + C** in the terminal where the server is running. You can restart it anytime with **npm run dev**.

4.3 What You Can and Cannot Test Locally

☐ Works Locally	☐ Won't Work Locally
All page layouts and styling	Form submissions (no backend)
Navigation between pages	Email sending
Content and text changes	QR code check-in
Responsive design (mobile/desktop)	Database operations
Countdown timer	Contact form submission
Animations and transitions	Any API calls

Since all the changes in this brief are frontend-only (content, layout, styling, new pages), everything you need to test will work locally.

5. The Daily Development Workflow

Follow this cycle every time you work on the project. It keeps your work safe and organized.

5.1 Before You Start Working

```
# 1. Pull the latest changes (in case anyone else pushed updates)
git checkout 2026-updates
git pull origin 2026-updates

# 2. Switch to your feature branch (or create a new one)
git checkout feature/your-task
# OR create a new one:
git checkout -b feature/new-task

# 3. Start the dev server
npm run dev
```

5.2 While You're Working

Make your changes in VS Code. The browser refreshes automatically. Commit frequently — think of commits like save points in a video game:

```
# See what files you've changed
git status

# See the actual changes (line by line)
git diff

# Stage specific files you want to commit
git add src/pages/Home.jsx
git add src/components/ui/HackathonSchedule.jsx

# Or stage everything
git add -A

# Commit with a clear message describing WHAT you changed
git commit -m "Update hero section with 2026 dates and countdown timer"

# Push to GitHub (backs up your work remotely)
git push
```

> **Good commit messages**
> Be specific: "Update hero section with 2026 dates and countdown timer" is much better than "changes" or "updated stuff". Future you will thank present you.

5.3 When You Finish a Task

```
# 1. Make sure everything is committed and pushed
git status  # should say "nothing to commit"
git push

# 2. Build the project to check for errors
```

```
npm run build

# 3. If the build succeeds, you're ready for a PR
```

If npm run build fails
Fix the errors before creating a PR. The build output will tell you exactly which file and line has the problem. Common issues: missing imports, typos in component names, or unclosed JSX tags.

6. Pull Requests — Getting Changes Reviewed and Deployed

A Pull Request (PR) is how you propose merging your changes into the main codebase. It creates a page on GitHub where you (and others) can review exactly what changed before it goes live.

6.1 Creating a Pull Request

6. Go to the repository on GitHub: https://github.com/Abrish-21/alx-hackathon-web
7. You'll likely see a yellow banner saying your branch had recent pushes with a "Compare & pull request" button. Click it.
8. If you don't see the banner: Click the "Pull requests" tab → "New pull request" → set base: main and compare: feature/your-branch
9. **Write a title:** Keep it short and descriptive, e.g., "Update all dates and venues to 2026"
10. **Write a description:** Explain what you changed and why. Reference the brief section number.
11. Click "Create pull request"

6.2 What Happens After You Create a PR

- **Vercel Preview Deployment:** If Vercel is configured (which it likely is), it will automatically build a preview version of your changes and post a link in the PR comments. This gives you a live URL to test your changes on.
- **Review your own PR:** Click the "Files changed" tab to see every line you modified. Read through it carefully. You'd be surprised how often you catch issues here.
- **Test the preview:** Click through every page on the preview URL. Check mobile view. Check all the links work.

6.3 Merging the PR

Once you're satisfied the changes look correct:

12. Click the green "Merge pull request" button on GitHub
13. Click "Confirm merge"
14. Delete the feature branch when prompted (keeps things clean)

After merging to main, Vercel will automatically deploy the changes to the live site. This typically takes 1–2 minutes.

> ⚠ **Deployment timing**
> After merging, wait 2–3 minutes then check the live site (hospitalityhackathon.et) to confirm the changes are live. If something looks wrong, you can revert the merge on GitHub.

7. Understanding the Project Structure

Knowing where things are in the project is essential. Here's a map of the files you'll be editing most:

7.1 Files You'll Edit Most

File	What It Controls
`src/App.jsx`	Router — add new routes here (e.g., /past-events)
`src/pages/Home.jsx`	The entire homepage — hero, sections, schedule
`src/pages/AboutUs.jsx`	About page with problem statements
`src/pages/Registration.jsx`	Registration form
`src/pages/Resources.jsx`	Resources page with articles, tools, FAQ
`src/pages/ContactUs.jsx`	Contact form (keep as-is per brief)
`src/pages/CheckIn.jsx`	QR check-in (keep as-is per brief)
`src/layouts/Navbar.jsx`	Top navigation bar — add new menu items here
`src/layouts/Footer.jsx`	Footer shown on every page
`src/components/ui/HackathonSchedule.jsx`	The schedule/agenda accordion
`src/components/ui/HackathonPeople.jsx`	Panelists/Judges cards
`src/components/ui/OrganizersShowcase.jsx`	Organizer logos section
`public/assets/images/`	All images used on the site

7.2 Adding a New Page

When you need to create the Past Events page (Section 5 of the brief):

15. Create a new file: src/pages/PastEvents.jsx
16. Add the route in src/App.jsx:

```
import PastEvents from './pages/PastEvents'

// Inside the <Routes> block, add:
<Route path='/past-events' element={<PastEvents />} />
```

17. Add a navigation link in src/layouts/Navbar.jsx:

```
// In the routes array at the top of the file, add:
{ path: "/past-events", label: "2025 Edition" },
```

8. Recommended Order of Work

Based on the brief, here's the order I'd suggest tackling the updates. This goes from foundational changes to more complex additions:

#	Task	Branch Name	Brief Section
1	Global date & venue updates	`feature/global-dates`	1.1
2	Homepage hero section	`feature/homepage-hero`	2.1
3	Homepage AI focus section	`feature/ai-focus-section`	2.3
4	Fireside chat + Judges + Partners	`feature/homepage-people`	2.4, 2.5, 2.6
5	About section spacing	`feature/about-spacing`	2.7
6	Problem statements update	`feature/problem-statements`	2.8
7	Agenda page rewrite	`feature/agenda-2026`	4
8	Registration page restructure	`feature/registration`	3.1, 3.2, 3.3
9	FAQ section	`feature/faq`	6
10	Past Events page (new)	`feature/past-events`	5
11	Training program + Resources	`feature/training-resources`	8
12	Navbar updates + final polish	`feature/nav-polish`	All

> **Start with #1**
> The global date updates touch every file and establish the foundation. Getting this right first means every subsequent task starts with correct dates already in place.

9. Common Git Situations and How to Handle Them

9.1 “I made changes on the wrong branch”

If you haven’t committed yet, you can move your uncommitted changes to the correct branch:

```
# Stash (temporarily save) your changes
git stash

# Switch to the correct branch
git checkout feature/correct-branch

# Apply your stashed changes
git stash pop
```

9.2 “I want to undo my last commit”

If you committed something but haven’t pushed it yet:

```
# Undo the commit but keep the changes in your files
git reset --soft HEAD~1
```

9.3 “I want to see what the live site looks like in my code”

Run a production build locally to see exactly what would be deployed:

```
npm run build
npm run preview
```

This builds the optimized version and serves it at a local URL.

9.4 “My terminal says there are merge conflicts”

This happens when two branches changed the same lines. Don’t panic:

18. Open the conflicting file in VS Code
19. Look for markers like **`<<<<<<< HEAD`**
20. The code between <<<<<<< and ======= is your version
21. The code between ======= and >>>>>>> is the other version
22. Delete the markers and keep the correct code
23. Save, then: git add the-file && git commit

10. Quick Reference — Commands You'll Use Daily

Command	What It Does
`npm run dev`	Start the local development server
`npm run build`	Build the project (check for errors)
`git status`	See what files have changed
`git diff`	See the actual line-by-line changes
`git add -A`	Stage all changes for commit
`git add <file>`	Stage a specific file
`git commit -m "message"`	Create a commit (save point)
`git push`	Push commits to GitHub
`git pull`	Download latest changes from GitHub
`git checkout <branch>`	Switch to a different branch
`git checkout -b <name>`	Create a new branch and switch to it
`git log --oneline -10`	See the last 10 commits
`git stash / git stash pop`	Temporarily save / restore changes
`Ctrl + C`	Stop the dev server

You're all set! Start with Step 1 (Prerequisites), and work through each section in order.
If you get stuck at any point, share the error message or screenshot and we'll work through it together.